

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 2

ENSEMBLE LEARNING IN PRACTICE

Reading Time 15 minutes

Prior Knowledge Bagging, Stacking, SVM, Decision Trees, Naïve Bayes, Logistic regression, ROC curve, AUC.

Keywords Base Learners, Meta Model.

In the previous lesson, we introduced two ensemble learning methods, namely bagging and stacking. This lesson will put these two methods into practice using Python.

1. Introduction

In this predictive problem, we apply ensemble methods to predict whether the Luxembourg index (LUXXX) exceeds a return of 0.25% in any direction. The predictors are a combination of country indices and technical indicators. Let's begin by importing the data.

```
In [ ]: import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
# Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

We create the target variable as before.

```
In [ ]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date", target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()

# Create Target Column.
```

```

# Shift period for target column
data_df[target_ETF + "_returns" + "_Shift"] = data_df[target_ETF + "_returns"].s

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target_ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)

```

Create the predictors below, namely four country indices and three technical indicators: slow to fast moving average ratio (SMA_ratio), relative strength index (RSI), and rate of change (RC).

```

In [ ]: # Four country indices used.
feats = ["MSCI KOREA", "MSCI DENMARK", "MSCI FRANCE", "MSCI NORWAY"]

# creating the technical indicators
data_df["SMA_5"] = data_df[target_ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target_ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)

# shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target_ETF] - data_df[target_ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up", "avg_15down"],
    axis=1,
    inplace=True,
)

data_df["RC"] = data_df[target_ETF].pct_change(15)

# all_feats
feats.append("SMA_ratio")

```

```
feats.append("RSI_ratio")
feats.append("RC")
```

Now that we have our data, we can apply the ensemble methods beginning with bagging.

2. Bagging with Random Forest

An example of a classifier that uses the bagging algorithm is the random forest classifier. Random forest is a tree-based algorithm; therefore, the weak learners are decision trees. The "random" applies more to the random features allocated to each decision tree; hence, not all features are used by the weak learners. The different subset of features and different samples used by the decision trees also result in uncorrelated trees, which improves the performance of the algorithm. The final outcome is simply a majority vote among the weak learners. For instance, if there are five decision trees in the random forest and three trees predict class 1 while two trees predict class 0, the random forest classification would be class 1 due to the majority prediction. We begin by importing the necessary libraries.

```
In [ ]: from sklearn import model_selection
        from sklearn.ensemble import RandomForestClassifier
```

Perform the train/test split of 80/20.

```
In [ ]: # Train/Test split
        # Train/Test split. No NaNs in the data.
        NoNaN_df = data_df.dropna()
        X = NoNaN_df[feats]

        X = X.iloc[:, :] # .values
        y = NoNaN_df.loc[:, "Target"] # .values

        del NoNaN_df

        # from sklearn.cross_validation import train_test_split
        X_train, X_test, y_train, y_test = model_selection.train_test_split(
            X, y, test_size=0.2, random_state=0
        )
```

We now create the random forest classifier. We call it our bag model since it is our bagging classifier. Among the hyperparameters specified is `n_estimators`, which specifies the number of trees. To reduce the time taken to train the classifier, we use a small number of 10 trees. A large number of trees will not lead to overfitting according to James (341).

```
In [ ]: bagmodel = RandomForestClassifier(n_estimators=10, random_state=4)
```

We train the model and look at its accuracy on the train and test set.

```
In [ ]: bagmodel.fit(X_train, y_train)
```

```
print("Accuracy on train set: %.4f" % (bagmodel.score(X_train, y_train)))
print("Accuracy on test set: %.4f" % (bagmodel.score(X_test, y_test)))
```

The big difference between the accuracies on the train and test set indicate over-fitting. We will look at optimal hyperparameter values to improve performance such as *min_samples_split*, which is the minimum number of samples required to split an internal node. We've mentioned *max_depth* in Lesson 3 of Module 3 for decision trees, which is the same for random forests. With limited computational power and quick turnaround times, we will explore just a few samples for these parameters. We use `GridSearchCV` as in Lesson 2 of Module 5 to find the optimal hyperparameters.

```
In [ ]: from sklearn.model_selection import GridSearchCV
```

```
In [ ]: # defining parameter range
param_grid = {
    "n_estimators": [10],
    "max_depth": [2, 3, 4],
    "min_samples_split": [2, 4, 8],
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=8), param_grid, refit=True, verbose=3, cv=5
)

# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
In [ ]: # print best parameter after tuning
print(grid.best_params_)

# print how our model looks after hyper-parameter tuning
print(grid.best_estimator_)
```

Apply optimal parameters and check if test score improves.

```
In [ ]: # Train with Tuned RF
# Create a tuned RF Classifier
bagmodel_tuned = RandomForestClassifier(
    n_estimators=10,
    max_depth=grid.best_params_["max_depth"],
    min_samples_split=grid.best_params_["min_samples_split"],
    random_state=8,
)

bagmodel_tuned.fit(X_train, y_train)
print("Accuracy on test set: %.4f" % (bagmodel_tuned.score(X_test, y_test)))
```

There is an uplift in accuracy for the test set by just exploring a small hyperparameter space. The small hyperparameter space is considered to keep computational time within reasonable limits. Students are encouraged to explore more hyperparameter options. Another measure of model performance to explore is the out-of-bag error, which is essentially the test error of a bagged model (James 342). Keep in mind that accuracy is based on the predicted classes, whereas the ROC AUC is based on predicted scores. We therefore plot the ROC Curve as in the previous modules to compare to the random

guess or no-skill model. From the output below, there is a clear benefit to the Random forest model over the no-skill model.

```
In [ ]: import matplotlib.pyplot as plt

# Performance
from sklearn.metrics import roc_auc_score, roc_curve

# predicted probabilities generated by tuned classifier
y_pred_proba = bagmodel_tuned.predict_proba(X_test)

# RF ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
auc = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)

# RF Model
plt.plot(fpr, tpr, label="RF, auc=" + str(auc))

# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```

Another useful feature from bagging is the *Out of Bag Score*. Refer to the video below for an explanation on this feature.

```
In [ ]: from IPython.display import VimeoVideo

VimeoVideo("785136013", h="2e436e22bd", width=600)
```

[Access Video Transcript here](#)

3. Stacking Model

The stacking model depends on the base models and meta model. For this illustration, we use the Gaussian Naïve Bayes, decision tree, and SVM classifiers as the base models. For the meta model, we use the simplistic logistic regression model. This is to show how we can leverage the logistic regression model using stacking. Let's import the necessary libraries for each of them. The names are well described and should be easy to match to the respective model.

```
In [ ]: from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
```

We define three base classifiers without any hyperparameter specifications, i.e., we use the default settings other than the kernel for SVC. This is to show the predictive power of

stacking without optimizing.

```
In [ ]: clf1 = DecisionTreeClassifier(random_state=2) # Decision Tree

clf2 = SVC(kernel="rbf", random_state=2) # Support Vector Classifier

clf3 = GaussianNB() # Gaussian Naive Bayes

est_rs = [("DTree", clf1), ("SVM", clf2), ("NB", clf3)]
```

Define the meta model.

```
In [ ]: mylr = LogisticRegression(random_state=2)

In [ ]: # creating a stacking classifier
stackingCLF = StackingClassifier(
    estimators=est_rs, final_estimator=mylr, stack_method="auto", cv=3
)
```

The stacking model is ready to go, and now, we need to train it on the training data. We also show the training score since we want to compare it to each of its base learners below as well.

```
In [ ]: stackingCLF.fit(X_train, y_train)
acc_score = stackingCLF.score(X_train, y_train)
round(acc_score, 4)

In [ ]: # Let's run the individual base learners and stacking `clf` to compare
for iterclf, iterlabel in zip([clf1, clf2, clf3], ["DTree", "SVM", "NB"]):
    scores = model_selection.cross_val_score(
        iterclf, X_train, y_train, cv=3, scoring="accuracy"
    )
    print("accuracy: %0.3f %s " % (scores.mean(), iterlabel))
```

It is quite evident that the stacking model outperforms all of its base learners and shows the benefit of ensemble learning in this case. It is worth comparing the performance of the two ensemble learning models used in this lesson; therefore, we look at the ROC curve and AUC of the stacking model versus the random forest model.

```
In [ ]: # predicted probabilities generated by tuned classifier
y_pred_probaStack = stackingCLF.predict_proba(X_test)

# Stacking Model ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_probaStack[:, 1])
auc = round(roc_auc_score(y_test, y_pred_probaStack[:, 1]), 4)

# RF ROC dependencies
fpr_RF, tpr_RF, _ = roc_curve(y_test, y_pred_proba[:, 1])
auc_RF = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)

# RF Model
plt.plot(fpr_RF, tpr_RF, label="RF, auc=" + str(auc_RF))
# Stacking Model
plt.plot(fpr, tpr, label="StackM, auc=" + str(auc))
```

```
# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```

From the plot above, it is clear that the stacking model slightly outperforms the random forest model.

4. Conclusion

This lesson looked at bagging and stacking in practice to predict a change of the LUXXX index by a certain threshold, i.e., a classification problem. Both these methods show added value over a no-skill model. In the next lesson, we will cover another ensemble learning method namely boosting and adaptive boosting.

References

James, Gareth et al. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed., Springer, 2021.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.